

Participation Section E:

Brief Intro

In this part, I will use Grok3 (fast) as a teacher to teach me about Initialization in deeplearning and its significant importance. I will try to let LLM to fully help me to understand this knowledge. I will attach my prompt and LLM's response (they are exported as a separated file), and my personal comment on LLM's response.

Comment

My first prompt is:

```
Hi there! I'm a student learning deep learning, and I want to learn about Initialization in Deep Learning. Could you first mathematically illustrate the necessity and significance of initialization?
```

As for the topic of initialization, my first prompt intend Grok to use math to illustrate why initialization is important and provide a formulation of the problem.

Grok exactly provide the scenario where bias and non-linearity is neglected, just like what instructor supposed in the lecture. Using elementary probability and the formulation of goal, Grok Successfully provided a landscape of the problem and prove the necessity of Initialization.

But the formulation of 'signals propagate through the network with stable magnitude' is still vague and this is crucial to understand why we emphasize initialization. My second prompt is:

What's the formulation of 'signals propagate through the network with stable magnitude', and why this is important?

Grok managed to give a detailed formulation about the signal magnitude stuff and help to grasp the intuitive idea about why we should initialize well. This is great. In the response of the first prompt, besides Xavier, Grok also mentioned He initialization. I want to know more about it. My third prompt it:

Thank you. You've mentioned about He initialization and the reason for this is ReLU halves the variance. Why the variance will be halved? What's the performance and theoretical difference between He and Xavier Initialization

Grok managed to convince me the ReLU scenario and proved that variance 'halving' theory. Till now, I'm content with Grok's response and know Initialization better. After lecture 6 and 7, I have a small question about whether the choice of norm will affect the choice of initialization. So my last prompt is:

Will the norm we use affect the initialization? For example, we choose from L2 norm to RMS norm.

Grok first explained what is RMS norm and briefly answered my question——yes. And he explained the reason mathematically and I'm convinced by what Grok said.

In summary, Grok indeed is a good tutor and can answer my questions in detail. I guess that this topic might has been covered in the corpus, so AI can handle my questions with ease. For these problems that they might be familiar with, AI rarely performs hallucination.

In the following part, the chat with Grok is attached.

Hi there! I'm a student learning deep learning, and I want to learn about Initialization in Deep Learning. Could you first mathematically illustrate the necessity and significance of initialization?

✓ Thought for 13s

Why Initialization Matters in Deep Learning

Initialization refers to the strategy for setting the initial values of weights (and sometimes biases) in a neural network before training begins. In deep learning, where networks can have many layers, poor initialization can lead to severe issues during training, such as vanishing or exploding gradients. These problems make optimization difficult or impossible, preventing the network from learning effectively. Proper initialization ensures that signals (activations and gradients) propagate through the network with stable magnitudes, allowing for efficient training.

To illustrate this mathematically, let's consider a simplified deep feedforward neural network without non-linear activations (for analytical tractability; the insights extend to non-linear cases). We'll analyze how the variance of activations propagates through layers and why uncontrolled variance leads to problems.

Setup: A Linear Deep Network

Suppose we have an L-layer linear network where the input is a vector $\mathbf{x} \in \mathbb{R}^{n_0}$, and each layer l (for $l = 1$ to L) computes:

$$\mathbf{a}_l = \mathbf{W}_l \mathbf{a}_{l-1},$$

with $\mathbf{a}_0 = \mathbf{x}$, and $\mathbf{W}_l \in \mathbb{R}^{n_l \times n_{l-1}}$ being the weight matrix for layer l . (We'll ignore biases for simplicity.)

Assume:

- The input $\mathbf{x}$ has zero mean and variance σ_x^2 for each component (i.e., $\mathbb{E}[x_i] = 0$, $\mathbb{V}[x_i] = \sigma_x^2$).
- Weights in $\mathbf{W}_l$ are independently initialized from a distribution with zero mean and variance σ_w^2 (e.g., Gaussian: $W_{ij} \sim \mathcal{N}(0, \sigma_w^2)$).

We want to track the variance of activations $\mathbb{V}[a_l^{(j)}]$ for the j -th neuron in layer l .

Forward Propagation: Variance of Activations

The activation for neuron j in layer l is:

$$a_l^{(j)} = \sum_{k=1}^{n_{l-1}} W_l^{(j,k)} a_{l-1}^{(k)}.$$

Since weights and previous activations are zero-mean and independent, the variance is:

$$\mathbb{V}[a_l^{(j)}] = \sum_{k=1}^{n_{l-1}} \mathbb{V}[W_l^{(j,k)} a_{l-1}^{(k)}] = \sum_{k=1}^{n_{l-1}} \mathbb{E}[(W_l^{(j,k)})^2] \cdot \mathbb{E}[(a_{l-1}^{(k)})^2] = n_{l-1} \cdot \sigma_w^2 \cdot \mathbb{V}[a_{l-1}^{(k)}],$$

assuming the variance is the same across neurons in a layer (by symmetry).

Let $\sigma_l^2 = \mathbb{V}[a_l^{(j)}]$. Then:

$$\sigma_l^2 = n_{l-1} \sigma_w^2 \sigma_{l-1}^2.$$

Recursively, starting from $\sigma_0^2 = \sigma_x^2$:

$$\sigma_l^2 = \sigma_x^2 \prod_{m=1}^l (n_{m-1} \sigma_w^2).$$

If we assume uniform layer widths $n = n_0 = n_1 = \dots = n_L$ for simplicity:

$$\sigma_l^2 = \sigma_x^2 (n \sigma_w^2)^l.$$

Now, consider the behavior as depth l increases (deep networks):

- If $n \sigma_w^2 > 1$ (e.g., $\sigma_w > 1/\sqrt{n}$), then $(n \sigma_w^2)^l$ grows exponentially with $l \rightarrow$ **exploding activations**: Outputs become extremely large, leading to numerical instability and overflow during training.
- If $n \sigma_w^2 < 1$ (e.g., $\sigma_w < 1/\sqrt{n}$), then $(n \sigma_w^2)^l$ shrinks exponentially to 0 $\rightarrow$ **vanishing activations**: Deep layers receive tiny inputs, making the network insensitive to the input data.
- If $n \sigma_w^2 = 1$ (e.g., $\sigma_w = 1/\sqrt{n}$), variance is preserved: $\sigma_l^2 = \sigma_x^2$ for all l , allowing stable signal propagation.

This shows the **necessity**: Without scaling σ_w properly (e.g., naively setting $\sigma_w = 1$ or a fixed small value), deep networks fail because activations either explode or vanish, rendering the model untrainable.

Backward Propagation: Variance of Gradients

Training uses backpropagation to compute gradients. For a loss $\mathcal{L}$, the gradient w.r.t. weights in layer l involves the chain rule. Simplifying to linear case, the gradient signal propagates backward similarly.

The partial derivative $\frac{\partial \mathcal{L}}{\partial \mathbf{a}_l} = \mathbf{W}_{l+1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{a}_{l+1}}$.

Assuming the top gradient $\frac{\partial \mathcal{L}}{\partial a_L^{(j)}}$ has variance $\sigma_{\nabla L}^2$, the backward variance recursion is analogous but in reverse:

$$\mathbb{V} \left[\frac{\partial \mathcal{L}}{\partial a_{l-1}^{(k)}} \right] = n_l \sigma_w^2 \mathbb{V} \left[\frac{\partial \mathcal{L}}{\partial a_l^{(j)}} \right].$$

Thus, similar exponential growth/decay occurs:

- If $n \sigma_w^2 > 1$, gradients explode backward $\rightarrow$ unstable updates, overshooting minima.
- If $n \sigma_w^2 < 1$, gradients vanish $\rightarrow$ early layers don't update, network doesn't learn from data.

This is the **vanishing/exploding gradient problem**, empirically observed in deep nets without proper init.

Significance: Incorporating Non-Linearity

In real networks, we add activations like sigmoid, tanh, or ReLU. Non-linearities exacerbate issues:

- Sigmoid/tanh squash inputs; if activations are large (exploding), outputs saturate near 0/1 or -1/1, where gradients are near zero $\rightarrow$ vanishing gradients.
- For ReLU, if variances are too small, many neurons "die" (output zero), blocking gradients.

Pioneering work (e.g., Glorot & Bengio, 2010) derived initializations to preserve variance:

- **Xavier/Glorot Initialization:** For tanh, set $\sigma_w^2 = \frac{1}{n_{in}}$ (forward) or average $\frac{2}{n_{in} + n_{out}}$ to balance forward/backward.
- **He Initialization:** For ReLU, $\sigma_w^2 = \frac{2}{n_{in}}$ (accounts for ReLU zeroing half the variance).

These ensure variances remain ~ 1 across layers, enabling training of very deep networks (e.g., ResNets with 100+ layers).

In summary, mathematically, initialization controls the scaling factor in variance propagation $(n \sigma_w^2)^l$. Proper choices prevent exponential instabilities, making deep

learning feasible. If you'd like, I can next cover common initialization methods, code examples, or experiments to demonstrate this!

What's the formulation of 'signals propagate through the network with stable magnitude', and why this is important?

Formulation of "Signals Propagate Through the Network with Stable Magnitude"

In deep learning, the phrase "signals propagate through the network with stable magnitude" refers to maintaining the variance (or scale) of activations (forward pass) and gradients (backward pass) at a consistent level as they traverse the layers of a neural network. Mathematically, this means ensuring that the variance of activations $\mathbb{V}[a_l]$ at layer l and the variance of gradients $\mathbb{V}[\frac{\partial \mathcal{L}}{\partial a_l}]$ remain approximately constant (or at least do not grow or shrink exponentially) across layers.

Let's formalize this for a linear feedforward network for clarity (insights extend to non-linear cases):

Forward Propagation (Activations)

Consider a neural network with L layers, where the activation at layer l is computed as:

$$\mathbf{a}_l = \mathbf{W}_l \mathbf{a}_{l-1},$$

where $\mathbf{a}_0 = \mathbf{x}$ (input), $\mathbf{W}_l \in \mathbb{R}^{n_l \times n_{l-1}}$ is the weight matrix, and no non-linearity is applied for simplicity. Assume:

- Input components have zero mean and variance $\mathbb{V}[x_i] = \sigma_x^2$.
- Weights $W_l^{(j,k)} \sim \mathcal{N}(0, \sigma_w^2)$ are i.i.d. with zero mean and variance σ_w^2 .

The variance of activations at layer l , $\sigma_l^2 = \mathbb{V}[a_l^{(j)}]$, for neuron j is:

$$\sigma_l^2 = \mathbb{V} \left[\sum_{k=1}^{n_{l-1}} W_l^{(j,k)} a_{l-1}^{(k)} \right] = \sum_{k=1}^{n_{l-1}} \mathbb{V}[W_l^{(j,k)} a_{l-1}^{(k)}] = n_{l-1} \sigma_w^2 \sigma_{l-1}^2.$$

Recursively, assuming uniform layer width $n = n_0 = n_1 = \dots = n_L$:

$$\sigma_l^2 = \sigma_x^2 (n \sigma_w^2)^l.$$

For stable magnitude, we want $\sigma_l^2 \approx \sigma_x^2$ for all layers l . This holds if:

$$n \sigma_w^2 = 1 \implies \sigma_w^2 = \frac{1}{n}.$$

This ensures the variance of activations remains roughly constant across layers, i.e., $\sigma_l^2 \approx \sigma_x^2$.

Backward Propagation (Gradients)

During backpropagation, the gradient of the loss $\mathcal{L}$ with respect to activations $\mathbf{a}_{l-1}$ is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_{l-1}} = \mathbf{W}_l^T \frac{\partial \mathcal{L}}{\partial \mathbf{a}_l}.$$

The variance of the gradient for neuron k in layer $l - 1$ is:

$$\mathbb{V} \left[\frac{\partial \mathcal{L}}{\partial a_{l-1}^{(k)}} \right] = \sum_{j=1}^{n_l} \mathbb{V} \left[W_l^{(j,k)} \frac{\partial \mathcal{L}}{\partial a_l^{(j)}} \right] = n_l \sigma_w^2 \mathbb{V} \left[\frac{\partial \mathcal{L}}{\partial a_l^{(j)}} \right].$$

Assuming uniform layer width n , the backward recursion mirrors the forward pass.

Starting from the top layer with $\mathbb{V} \left[\frac{\partial \mathcal{L}}{\partial a_L^{(j)}} \right] = \sigma_{\nabla L}^2$, we get:

$$\mathbb{V} \left[\frac{\partial \mathcal{L}}{\partial a_{l-1}^{(j)}} \right] = (n \sigma_w^2) \mathbb{V} \left[\frac{\partial \mathcal{L}}{\partial a_l^{(j)}} \right].$$

After $L - l$ steps backward, the variance is:

$$\mathbb{V} \left[\frac{\partial \mathcal{L}}{\partial a_{l-1}^{(j)}} \right] = \sigma_{\nabla L}^2 (n \sigma_w^2)^{L-l}.$$

For stable gradient magnitude, we again want $\mathbb{V} \left[\frac{\partial \mathcal{L}}{\partial a_{l-1}^{(j)}} \right] \approx \sigma_{\nabla L}^2$, which requires:

$$n \sigma_w^2 = 1 \implies \sigma_w^2 = \frac{1}{n}.$$

This ensures gradients neither explode nor vanish as they propagate backward.

Non-Linearities

In practice, layers include non-linear activations (e.g., ReLU, tanh). These affect variance:

- **ReLU:** Outputs are zero for negative inputs, halving the variance (since half the inputs are zeroed). To compensate, He initialization uses $\sigma_w^2 = \frac{2}{n_{in}}$.
- **Tanh/Sigmoid:** These squash outputs, so Xavier/Glorot initialization balances forward and backward passes with $\sigma_w^2 = \frac{2}{n_{in} + n_{out}}$ or $\sigma_w^2 = \frac{1}{n_{in}}$.

Stable magnitude means:

$$\sigma_l^2 \approx \text{constant}, \quad \forall \left[\frac{\partial \mathcal{L}}{\partial a_l^{(j)}} \right] \approx \text{constant}.$$

Why This Is Important

Maintaining stable signal magnitudes is critical for effective training of deep neural networks. Here's why:

1. Prevents Vanishing/Exploding Gradients:

- If $n\sigma_w^2 > 1$, variances grow exponentially $((n\sigma_w^2)^l) \rightarrow$ **exploding signals**. Activations and gradients become too large, causing numerical instability or divergent optimization.
- If $n\sigma_w^2 < 1$, variances shrink exponentially $\rightarrow$ **vanishing signals**. Deep layers receive tiny activations/gradients, so early layers barely learn, stalling training.
- Stable variances ($n\sigma_w^2 \approx 1$) ensure gradients remain in a reasonable range, enabling consistent learning across all layers.

2. Ensures Effective Signal Propagation:

- Forward: Stable activation magnitudes prevent inputs to neurons from becoming too small (losing information) or too large (saturating non-linearities like tanh, where gradients vanish).
- Backward: Stable gradients ensure all layers receive meaningful updates, allowing the network to learn features from data effectively.

3. Enables Training Deep Networks:

- Without stable signal propagation, deep networks (e.g., 50+ layers) are nearly impossible to train due to vanishing/exploding issues. Proper initialization (e.g., Xavier, He) allows training of architectures like ResNets or Transformers with hundreds of layers.

4. Improves Convergence Speed:

- Stable signals lead to balanced updates during optimization, avoiding slow convergence (from tiny gradients) or instability (from large gradients). This reduces training time and improves final performance.

5. Handles Non-Linear Effects:

- Non-linearities like ReLU or tanh can amplify vanishing/exploding issues. For example, ReLU zeros negative inputs, reducing variance, while tanh saturates

for large inputs, flattening gradients. Proper initialization accounts for these effects to maintain signal stability.

Practical Implication

Initialization methods like **Xavier/Glorot** ($\sigma_w^2 = \frac{1}{n_{in}}$ or $\frac{2}{n_{in}+n_{out}}$) or **He** ($\sigma_w^2 = \frac{2}{n_{in}}$) are derived to achieve this stability by setting σ_w^2 such that $n\sigma_w^2 \approx 1$ (adjusted for nonlinearities). For example:

- Xavier ensures stable variance for tanh/sigmoid.
- He adjusts for ReLU's variance reduction.

In code (e.g., PyTorch), this looks like:

python

```
import torch.nn as nn
# Xavier for tanh
nn.init.xavier_uniform_(layer.weight)
# He for ReLU
nn.init.kaiming_uniform_(layer.weight, nonlinearity='relu')
```

Conclusion

The formulation $\sigma_l^2 \approx \sigma_x^2$ and $\mathbb{V}[\frac{\partial \mathcal{L}}{\partial a_l}] \approx \text{constant}$ ensures signals maintain stable magnitudes. This is crucial to prevent vanishing/exploding gradients, enable deep network training, ensure effective learning, and improve convergence. If you'd like, I can dive into specific initialization schemes, provide code to simulate variance propagation, or discuss how this applies to modern architectures like Transformers!

Thank you. You've mentioned about He initialization and the reason for this is ReLU halves the variance. Why the variance will be halved? What's the performance and theoretical difference between He and Xavier Initialization

To address your questions, let's dive into why ReLU halves the variance of activations, and then compare the performance and theoretical differences between **He initialization** and **Xavier/Glorot initialization**. I'll keep the explanation clear, concise, and mathematically grounded, focusing on your request about variance and the comparison between these initialization methods.

Why ReLU Halves the Variance

The ReLU (Rectified Linear Unit) activation function is defined as:

$$f(z) = \max(0, z).$$

This means that any negative input is set to zero, while positive inputs pass through unchanged. When analyzing the effect of ReLU on the variance of activations, we need to consider how it transforms the distribution of the pre-activation values.

Assumptions

- Suppose the pre-activation values $z_l^{(j)}$ at layer l (i.e., $z_l = \mathbf{W}_l \mathbf{a}_{l-1}$) are zero-mean with variance $\mathbb{V}[z_l^{(j)}] = \sigma_z^2$.
- For simplicity, assume $z_l^{(j)} \sim \mathcal{N}(0, \sigma_z^2)$, a Gaussian distribution, which is reasonable due to the central limit theorem when summing many weighted inputs (i.e., $z_l^{(j)} = \sum_k W_l^{(j,k)} a_{l-1}^{(k)}$).

Effect of ReLU

The ReLU function $a_l^{(j)} = \max(0, z_l^{(j)})$ transforms the distribution:

- If $z_l^{(j)} < 0$, then $a_l^{(j)} = 0$.
- If $z_l^{(j)} \geq 0$, then $a_l^{(j)} = z_l^{(j)}$.

Since $z_l^{(j)}$ is Gaussian and symmetric around zero, half of the values ($P(z_l^{(j)} < 0) = 0.5$) are set to zero, and the other half ($P(z_l^{(j)} \geq 0) = 0.5$) retain their original values. To compute the variance of the output $a_l^{(j)}$, we use the formula for the variance of a truncated distribution.

The variance of the ReLU output $a_l^{(j)} = \max(0, z_l^{(j)})$ can be derived as follows:

- For a zero-mean Gaussian $z \sim \mathcal{N}(0, \sigma_z^2)$, the ReLU output is a mixture of a point mass at 0 (with probability 0.5) and a half-Gaussian for $z \geq 0$.
- The variance of $a = \max(0, z)$ is:

$$\mathbb{V}[a] = \mathbb{E}[a^2] - (\mathbb{E}[a])^2.$$

- **Compute $\mathbb{E}[a]$:**

$$\mathbb{E}[a] = \int_0^\infty z \cdot \frac{1}{\sqrt{2\pi\sigma_z^2}} e^{-z^2/(2\sigma_z^2)} dz.$$

Substitute $u = z/\sigma_z$, so $z = \sigma_z u$, $dz = \sigma_z du$:

$$\mathbb{E}[a] = \sigma_z \int_0^\infty u \cdot \frac{1}{\sqrt{2\pi}} e^{-u^2/2} du = \sigma_z \cdot \frac{1}{\sqrt{2\pi}} \int_0^\infty u e^{-u^2/2} du.$$

The integral evaluates to 1 (since $\int_0^\infty u e^{-u^2/2} du = [-e^{-u^2/2}]_0^\infty = 1$):

$$\mathbb{E}[a] = \sigma_z \cdot \frac{1}{\sqrt{2\pi}}.$$

- **Compute $\mathbb{E}[a^2]$:**

$$\mathbb{E}[a^2] = \int_0^\infty z^2 \cdot \frac{1}{\sqrt{2\pi\sigma_z^2}} e^{-z^2/(2\sigma_z^2)} dz = \sigma_z^2 \int_0^\infty u^2 \cdot \frac{1}{\sqrt{2\pi}} e^{-u^2/2} du.$$

The integral $\int_0^\infty u^2 e^{-u^2/2} du = \sqrt{\pi/2}$, so:

$$\mathbb{E}[a^2] = \sigma_z^2 \cdot \frac{1}{\sqrt{2\pi}} \cdot \sqrt{\frac{\pi}{2}} = \sigma_z^2 \cdot \frac{1}{2}.$$

- **Variance:**

$$\mathbb{V}[a] = \mathbb{E}[a^2] - (\mathbb{E}[a])^2 = \frac{\sigma_z^2}{2} - \left(\frac{\sigma_z}{\sqrt{2\pi}} \right)^2 = \frac{\sigma_z^2}{2} - \frac{\sigma_z^2}{2\pi}.$$

For large σ_z , the second term is small, so:

$$\mathbb{V}[a] \approx \frac{\sigma_z^2}{2}.$$

Thus, ReLU approximately **halves the variance** of the input distribution because it zeros out half the values, reducing the spread of the output distribution.

Implication

If the pre-activation variance is $\sigma_z^2 = n_{in}\sigma_w^2\sigma_{l-1}^2$ (from the linear transformation $\mathbf{W}_l\mathbf{a}_{l-1}$), the post-ReLU variance is:

$$\mathbb{V}[a_l^{(j)}] \approx \frac{1}{2} n_{in}\sigma_w^2\sigma_{l-1}^2.$$

To maintain $\mathbb{V}[a_l^{(j)}] \approx \mathbb{V}[a_{l-1}^{(j)}]$, we need:

$$\frac{1}{2} n_{in}\sigma_w^2 \approx 1 \implies \sigma_w^2 = \frac{2}{n_{in}}.$$

This is the basis for **He initialization**, which compensates for ReLU's variance reduction.

He vs. Xavier Initialization: Theoretical and Performance Differences

Theoretical Differences

1. Xavier/Glorot Initialization:

- **Designed for:** Symmetric activations like tanh or sigmoid, which have zero-mean outputs and bounded ranges (e.g., $[-1, 1]$ for tanh).
- **Formulation:**
 - Weights are initialized with variance:

$$\sigma_w^2 = \frac{2}{n_{in} + n_{out}} \quad \text{or} \quad \sigma_w^2 = \frac{1}{n_{in}},$$

where n_{in} is the number of input units, and n_{out} is the number of output units.

- Uniform distribution: $W \sim \text{Uniform}\left(-\sqrt{\frac{6}{n_{in}+n_{out}}}, \sqrt{\frac{6}{n_{in}+n_{out}}}\right)$.
- **Rationale:** Balances forward and backward signal propagation for tanh/sigmoid, where the activation function doesn't significantly alter the variance (tanh is near-linear around zero). The goal is to keep $\mathbb{V}[a_l] \approx \mathbb{V}[a_{l-1}]$ and $\mathbb{V}[\frac{\partial \mathcal{L}}{\partial a_l}] \approx \text{constant}$.
- **Assumption:** The activation function is roughly linear near the origin, and forward/backward variances should be balanced (hence the averaging of n_{in} and n_{out}).

2. He Initialization:

- **Designed for:** ReLU and its variants (e.g., Leaky ReLU), which are non-negative and zero out negative inputs.
- **Formulation:**
 - Weights are initialized with variance:

$$\sigma_w^2 = \frac{2}{n_{in}}.$$

- Uniform distribution: $W \sim \text{Uniform}\left(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}}\right)$.
- **Rationale:** Accounts for ReLU's variance reduction (halving due to zeroing negative inputs). By setting $\sigma_w^2 = \frac{2}{n_{in}}$, the post-ReLU variance is:

$$\mathbb{V}[a_l] \approx \frac{1}{2} \cdot n_{in} \cdot \frac{2}{n_{in}} \cdot \mathbb{V}[a_{l-1}] = \mathbb{V}[a_{l-1}],$$

ensuring stable forward propagation. For backward propagation, a similar scaling ensures gradient stability.

- **Assumption:** Only forward propagation variance is considered primarily, as ReLU's non-linearity affects the forward pass more significantly, and n_{in} dominates in deep networks.

3. Key Theoretical Difference:

- **Xavier** assumes a symmetric activation function (tanh/sigmoid) and balances input/output layer sizes to stabilize both forward and backward passes. It uses $\sigma_w^2 \propto \frac{1}{n_{in} + n_{out}}$ to compromise between forward ($\propto 1/n_{in}$) and backward ($\propto 1/n_{out}$) variance preservation.
- **He** is tailored for ReLU, where the variance is halved in the forward pass. It uses a larger variance ($\sigma_w^2 = \frac{2}{n_{in}}$) to compensate, focusing primarily on the forward pass and input size, as ReLU's effect is more pronounced there.
- Mathematically:
 - Xavier: $\mathbb{V}[z_l] = n_{in} \sigma_w^2 \mathbb{V}[a_{l-1}] \approx \mathbb{V}[a_{l-1}]$ (tanh preserves variance near zero).
 - He: $\mathbb{V}[a_l] \approx \frac{1}{2} n_{in} \sigma_w^2 \mathbb{V}[a_{l-1}] \approx \mathbb{V}[a_{l-1}]$ (ReLU halves variance, so σ_w^2 is doubled).

Performance Differences

1. With ReLU-Based Networks:

- **He Initialization:**
 - **Better Performance:** He initialization is specifically designed for ReLU and its variants. By compensating for the variance reduction, it ensures activations and gradients remain in a healthy range, preventing issues like "dead neurons" (where ReLU units always output zero due to negative pre-activations) and vanishing gradients.
 - **Empirical Evidence:** In deep networks (e.g., ResNets, VGG) with ReLU, He initialization leads to faster convergence and better final accuracy compared to Xavier. For example, He et al. (2015) showed that deep networks with ReLU failed to train effectively with Xavier due to insufficient activation variance, while He initialization enabled training of 100+ layer networks.

- **Xavier Initialization:**

- **Poorer Performance:** When used with ReLU, Xavier underestimates the variance needed ($\sigma_w^2 = \frac{1}{n_{in}}$ or less). This leads to shrinking activations in deep layers ($\mathbb{V}[a_l] \approx \frac{1}{2}n_{in} \cdot \frac{1}{n_{in}}\mathbb{V}[a_{l-1}] = \frac{1}{2}\mathbb{V}[a_{l-1}]$), causing vanishing gradients and slower learning or convergence to suboptimal solutions.
- **Dead Neurons:** Smaller weights increase the chance of negative pre-activations, causing more ReLU units to output zero, reducing the network's capacity.

2. With Tanh/Sigmoid-Based Networks:

- **Xavier Initialization:**

- **Better Performance:** Xavier is optimal for tanh/sigmoid, as it maintains stable variances for these symmetric, bounded activations. It prevents saturation (where large inputs push tanh/sigmoid to flat regions with near-zero gradients), enabling effective training.
- **Empirical Evidence:** Glorot & Bengio (2010) showed that Xavier initialization significantly improved training of deep networks with tanh/sigmoid compared to random or uniform initializations with improper scales.

- **He Initialization:**

- **Poorer Performance:** He uses a larger variance ($\sigma_w^2 = \frac{2}{n_{in}}$), which can lead to overly large activations for tanh/sigmoid. This pushes activations toward saturation regions (e.g., ± 1 for tanh), where gradients are small, slowing learning or causing instability.
- **Overcompensation:** The doubled variance is unnecessary for tanh, which doesn't halve variance like ReLU, leading to potential exploding activations in early training.

3. Modern Architectures:

- Most modern deep learning models (e.g., CNNs, ResNets, Transformers) use ReLU or its variants (Leaky ReLU, GELU) due to faster training and better handling of sparse gradients. **He initialization** is the default choice in frameworks like PyTorch and TensorFlow for ReLU-based layers.
- Xavier remains relevant for specific cases, like RNNs (e.g., LSTMs) or Transformers with tanh activations, but these are less common in modern architectures.

Practical Example (PyTorch)

python

```
import torch
import torch.nn as nn

# He Initialization for ReLU
conv = nn.Conv2d(3, 64, kernel_size=3)
nn.init.kaiming_normal_(conv.weight, nonlinearity='relu') # He init: var

# Xavier Initialization for Tanh
fc = nn.Linear(512, 256)
nn.init.xavier_normal_(fc.weight) # Xavier init: var = 2/(n_in + n_out)
```

Summary

- **Why ReLU Halves Variance:** ReLU zeros out negative inputs, which constitute half of a zero-mean Gaussian distribution, reducing the output variance to approximately $\frac{1}{2}\sigma_z^2$. He initialization compensates by setting $\sigma_w^2 = \frac{2}{n_{in}}$.
- **He vs. Xavier:**
 - **Theoretical:** He is tailored for ReLU, doubling the variance to counter its halving effect, focusing on forward propagation. Xavier balances forward/backward variances for symmetric activations like tanh, using a smaller variance.
 - **Performance:** He outperforms Xavier for ReLU-based networks (faster convergence, fewer dead neurons), while Xavier is better for tanh/sigmoid networks (avoids saturation). In modern ReLU-dominated architectures, He is preferred.
- **Practical Tip:** Use He for ReLU-based networks (CNNs, Transformers) and Xavier for tanh/sigmoid-based networks (some RNNs). Always match the initialization to the activation function.

If you'd like, I can provide a code simulation to demonstrate the variance halving with ReLU or compare He vs. Xavier empirically on a small network!

Will the norm we use affect the initialization? For example, we choose from L2 norm to RMS norm.

The choice of norm used in analyzing or constraining the initialization of weights in a neural network can indeed affect the initialization strategy, as it influences how we measure and control the "magnitude" or "scale" of the weights. The norm defines the way we quantify the size of the weight vectors or matrices, which in turn impacts the variance or scaling of weights during initialization. Below, I'll explain how different norms (e.g., L2 norm vs. RMS norm) affect initialization, with a focus on their implications for signal propagation in deep learning. I'll keep the explanation mathematical yet accessible, addressing your question directly.

Norms and Their Role in Initialization

In deep learning, weight initialization aims to ensure stable signal propagation (activations and gradients) through the network, as discussed previously. The "magnitude" of weights is often controlled by setting their variance or scaling them to satisfy a specific norm. The norm used to measure this magnitude can influence the initialization process, as different norms emphasize different aspects of the weight distribution.

- **L2 Norm:** The L2 norm (Euclidean norm) of a weight vector $\mathbf{w} = [w_1, w_2, \dots, w_n]$ is:

$$\|\mathbf{w}\|_2 = \sqrt{\sum_{i=1}^n w_i^2}.$$

For a weight matrix $\mathbf{W} \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$, the L2 norm (or Frobenius norm) is:

$$\|\mathbf{W}\|_F = \sqrt{\sum_{i,j} W_{ij}^2}.$$

The L2 norm measures the total magnitude of weights, emphasizing larger values due to the squaring operation.

- **RMS Norm** (Root Mean Square norm): For a weight vector, the RMS norm is:

$$\text{RMS}(\mathbf{w}) = \sqrt{\frac{1}{n} \sum_{i=1}^n w_i^2} = \frac{\|\mathbf{w}\|_2}{\sqrt{n}}.$$

For a matrix, it's similarly defined over all elements:

$$\text{RMS}(\mathbf{W}) = \sqrt{\frac{1}{n_{\text{in}} n_{\text{out}}} \sum_{i,j} W_{ij}^2}.$$

The RMS norm is essentially the L2 norm normalized by the number of elements, representing the "average" magnitude per weight. It's closely related to the standard deviation of the weights (since $\text{RMS}^2 = \mathbb{E}[w_i^2]$ for zero-mean weights).

In initialization, we typically aim to control the variance of weights ($\mathbb{V}[w_i] = \sigma_w^2$) or their norm to ensure stable signal propagation. The choice of norm affects how we set σ_w^2 or scale the weights, as it defines the metric for "stability."

How Norms Affect Initialization

To understand the impact, let's revisit the goal of initialization: maintaining stable variances of activations and gradients across layers. For a linear layer $\mathbf{a}_l = \mathbf{W}_l \mathbf{a}_{l-1}$, the variance of activations is:

$$\mathbb{V}[\mathbf{a}_l^{(j)}] = n_{\text{in}} \sigma_w^2 \mathbb{V}[\mathbf{a}_{l-1}^{(k)}],$$

where $\sigma_w^2 = \mathbb{V}[W_{ij}]$ is the variance of the weights. For stable propagation, we want $\mathbb{V}[\mathbf{a}_l] \approx \mathbb{V}[\mathbf{a}_{l-1}]$, which requires:

$$n_{\text{in}} \sigma_w^2 \approx 1 \implies \sigma_w^2 = \frac{1}{n_{\text{in}}}.$$

For ReLU, the variance is halved post-activation, so He initialization uses $\sigma_w^2 = \frac{2}{n_{\text{in}}}$.

The norm we use influences how we enforce or interpret this variance:

- **L2 Norm:** If we constrain the L2 norm of the weight vector per neuron (row of $\mathbf{W}_l$), we're controlling $\|\mathbf{w}_j\|_2^2 = \sum_{k=1}^{n_{\text{in}}} W_{jk}^2$. For i.i.d. weights with variance σ_w^2 , the expected L2 norm squared is:

$$\mathbb{E}[\|\mathbf{w}_j\|_2^2] = \mathbb{E}\left[\sum_{k=1}^{n_{\text{in}}} W_{jk}^2\right] = n_{\text{in}} \sigma_w^2.$$

To achieve $\sigma_w^2 = \frac{1}{n_{\text{in}}}$ (Xavier) or $\frac{2}{n_{\text{in}}}$ (He), we set the L2 norm such that:

$$\mathbb{E}[\|\mathbf{w}_j\|_2^2] = n_{\text{in}} \cdot \frac{1}{n_{\text{in}}} = 1 \quad (\text{Xavier}),$$

or

$$\mathbb{E}[\|\mathbf{w}_j\|_2^2] = n_{\text{in}} \cdot \frac{2}{n_{\text{in}}} = 2 \quad (\text{He}).$$

Thus, L2 norm constraints translate directly to variance control, as σ_w^2 scales with the number of inputs.

- **RMS Norm:** The RMS norm is directly tied to the standard deviation:

$$\text{RMS}(\mathbf{w}_j) = \sqrt{\frac{1}{n_{\text{in}}} \sum_{k=1}^{n_{\text{in}}} W_{jk}^2} = \sqrt{\mathbb{E}[W_{jk}^2]} = \sigma_w \quad (\text{for zero-mean weights}).$$

If we constrain the RMS norm of the weights to a specific value, say $\text{RMS}(\mathbf{w}_j) = c$, then:

$$\sigma_w = c \implies \sigma_w^2 = c^2.$$

To match Xavier ($\sigma_w^2 = \frac{1}{n_{\text{in}}}$) or He ($\sigma_w^2 = \frac{2}{n_{\text{in}}}$), we set:

$$\sigma_w = \sqrt{\frac{1}{n_{\text{in}}}} \quad (\text{Xavier}) \quad \text{or} \quad \sigma_w = \sqrt{\frac{2}{n_{\text{in}}}} \quad (\text{He}).$$

The RMS norm is equivalent to controlling the standard deviation directly, making it a natural fit for variance-based initialization like Xavier or He.

Key Difference in Initialization

- **L2 Norm:** When initializing with an L2 norm constraint, we control the total magnitude of the weight vector, which scales with $\sqrt{n_{\text{in}}\sigma_w^2}$. For example, in Xavier initialization, we might scale weights so that $\|\mathbf{w}_j\|_2 \approx \sqrt{1}$ or adjust based on n_{in} . This is common in uniform distributions (e.g., Xavier uniform: $W \sim \text{Uniform}(-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}})$).
- **RMS Norm:** When using the RMS norm, we directly control σ_w , the standard deviation of individual weights. This aligns closely with variance-based analyses (e.g., He: $\sigma_w = \sqrt{\frac{2}{n_{\text{in}}}}$). RMS is more intuitive for Gaussian initializations, as it directly corresponds to the weight distribution's spread.

In practice, both norms can achieve the same initialization (e.g., Xavier or He) by adjusting the scaling factor, but the formulation differs:

- **L2:** $\|\mathbf{w}_j\|_2 \approx \sqrt{n_{\text{in}}\sigma_w^2}$, so the norm grows with the number of inputs.
- **RMS:** $\text{RMS}(\mathbf{w}_j) \approx \sigma_w$, independent of n_{in} in terms of the per-weight scale.

Practical Implications

1. Effect on Initialization Schemes:

- **Xavier Initialization:**

- **L2 Norm:** In PyTorch's `xavier_uniform_`, weights are scaled to ensure the L2 norm of the weight vector aligns with $\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}$, which corresponds to $\sigma_w^2 = \frac{2}{n_{\text{in}}+n_{\text{out}}}$.
- **RMS Norm:** If using RMS, we set $\sigma_w = \sqrt{\frac{2}{n_{\text{in}}+n_{\text{out}}}}$, directly controlling the standard deviation. This is equivalent for Gaussian initializations (e.g., `xavier_normal_`).
- **He Initialization:**
 - **L2 Norm:** Weights are scaled so that $\|\mathbf{w}_j\|_2 \approx \sqrt{2}$ for ReLU (since $\sigma_w^2 = \frac{2}{n_{\text{in}}} \implies \mathbb{E}[\|\mathbf{w}_j\|_2^2] = n_{\text{in}} \cdot \frac{2}{n_{\text{in}}} = 2$).
 - **RMS Norm:** Set $\sigma_w = \sqrt{\frac{2}{n_{\text{in}}}}$, as in `kaiming_normal_` in PyTorch, directly matching the variance needed for ReLU.

2. Impact on Signal Propagation:

- Both norms aim to achieve $\mathbb{V}[a_l] \approx \mathbb{V}[a_{l-1}]$ and stable gradients. The choice of norm affects how we compute or enforce this:
 - **L2 Norm:** Emphasizes the total magnitude of the weight vector, which may lead to slightly different scaling in practice, especially for non-uniform or sparse weight distributions.
 - **RMS Norm:** Directly controls per-weight variance, making it more straightforward for Gaussian or variance-based initializations.
- For ReLU, the RMS norm is particularly intuitive because He initialization explicitly sets $\sigma_w^2 = \frac{2}{n_{\text{in}}}$ to counter the variance halving, and $\text{RMS} = \sigma_w$ aligns with this goal.

3. Performance Differences:

- **Minimal Difference in Standard Cases:** For standard initialization schemes (Xavier, He) implemented correctly, whether you use L2 or RMS norm to derive the scaling, the resulting weight distribution is equivalent (e.g., same σ_w^2). Thus, performance is typically identical for well-tuned initializations.
- **Sparse or Non-Standard Distributions:** If weights are sparse (e.g., in sparse networks) or follow non-Gaussian distributions, the L2 norm might overemphasize large weights, while RMS norm averages across all weights. RMS may be more robust in such cases, as it normalizes by the number of elements.

- **Numerical Stability:** RMS norm is less sensitive to the dimensionality n_{in} in terms of per-weight scaling, which can be advantageous in very high-dimensional layers (e.g., Transformers with large fan-in).

4. Implementation in Frameworks:

- **PyTorch/TensorFlow:** Most implementations (e.g., `kaiming_normal_`, `xavier_uniform_`) implicitly use variance-based scaling, which aligns with RMS norm thinking (σ_w). However, uniform distributions (e.g., `xavier_uniform_`) are derived to match the L2 norm of the weight vector to the desired variance.
- Example:

python

```
import torch
import torch.nn as nn
layer = nn.Linear(512, 256)
# He initialization (RMS norm implicit in variance)
nn.init.kaiming_normal_(layer.weight, nonlinearity='relu') # Sets
# Xavier uniform (L2 norm implicit in range)
nn.init.xavier_uniform_(layer.weight) # Sets range ~ sqrt(6/(512+256))
```

Theoretical Insights

- **L2 Norm:**
 - Pros: Naturally captures the total magnitude of the weight vector, which is relevant for matrix operations and signal propagation (since $\mathbf{a}_l = \mathbf{W}_l \mathbf{a}_{l-1}$ involves the entire weight vector).
 - Cons: Scales with $\sqrt{n_{\text{in}}}$, which can lead to larger norms in high-dimensional layers, potentially requiring careful normalization.
 - Use Case: Common in uniform initializations or when constraining the Frobenius norm of weight matrices.
- **RMS Norm:**
 - Pros: Directly corresponds to σ_w , making it intuitive for variance-based analyses (e.g., He, Xavier). Normalizes by the number of elements, so it's less sensitive to layer size.

- **Cons:** Less emphasis on the total magnitude of the weight vector, which might matter in certain architectures (e.g., when spectral norm matters).
 - **Use Case:** Preferred for Gaussian initializations or when focusing on per-weight variance.
 - **Equivalence in Practice:** For i.i.d. weights, the relationship $\text{RMS} = \frac{\|\mathbf{w}\|_2}{\sqrt{n}}$ means that setting the RMS norm to $\sqrt{\frac{2}{n_{\text{in}}}}$ (He) or $\sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}$ (Xavier) achieves the same variance as L2-based scaling. The choice of norm is thus more about the formulation than the outcome, unless dealing with non-standard distributions.
-

Conclusion

The choice of norm (L2 vs. RMS) affects how we formulate and enforce weight initialization, but for standard schemes like Xavier and He, the resulting weight distributions are equivalent when properly scaled:

- **L2 Norm:** Controls the total magnitude of the weight vector, scaling with $\sqrt{n_{\text{in}}\sigma_w^2}$. Common in uniform initializations.
- **RMS Norm:** Directly controls the standard deviation (σ_w), aligning with variance-based analyses. Natural for Gaussian initializations.
- **Impact:** Both can achieve stable signal propagation (e.g., $\mathbb{V}[a_l] \approx \mathbb{V}[a_{l-1}]$) if σ_w^2 is set correctly (e.g., $\frac{2}{n_{\text{in}}}$ for He, $\frac{2}{n_{\text{in}} + n_{\text{out}}}$ for Xavier). Performance differences are minimal for standard cases but may arise in sparse or non-Gaussian settings, where RMS is often more robust.
- **Practical Note:** Modern frameworks use variance-based scaling (implicitly RMS-like) for Gaussian initializations and L2-derived ranges for uniform initializations, but the goal is the same: stable variances.

If you'd like, I can provide a code example to simulate how L2 vs. RMS norm constraints affect weight distributions or demonstrate their impact on a small network's training!